Ejercicios_Repaso_PLSQL

```
# Ejercicios de Repaso — PL/SQL

>Ejercicios progresivos basados en las clases de la unidad de PL/SQL.
>
> **Modalidad:** resolver primero sin mirar soluciones. La idea es avanzar desde ejercicios simples hacia problemas que combinen varios conceptos.

---

## Clase 1 — Bloques anónimos

### Ejercicio 1 — Información básica

Crea un bloque anónimo que tenga:

- Una variable `v_nombre` de tipo `VARCHAR2`.
- Una variable `v_edad` de tipo `NUMBER`.
- Una variable `v_ciudad` de tipo `VARCHAR2`.

Asigna valores directamente a las variables y muestra mediante `DBMS_OUTPUT.PUT_LINE`:

```text
Nombre: Juan
Edad: 25
Ciudad: Santiago
```
```

Practica: `DECLARE`,variables,asignación y `DBMS_OUTPUT`.

Resultado

```plsql
DECLARE
    v_nombre VARCHAR2(30) := 'Juan';
    v_edad number(2) := 25;
    v_ciudad VARCHAR2(30) := 'Santiago';

BEGIN
    dbms_output.put_line('Nombre: ' ||v_nombre);
    dbms_output.put_line('Edad: ' ||v_edad);
    dbms_output.put_line('Ciudad: ' ||v_ciudad);
END;
```

Ejercicio 2 — Calculadora con condición

Crea un bloque anónimo que solicite mediante variables de sustitución:

```text
Número 1:
Número 2:
```

El programa debe calcular:

- Suma.
- Resta.
- Multiplicación.
- División.

Además, debe determinar cuál de los dos números es mayor.

Ejemplo:

```text
Número 1: 20
Número 2: 5

Suma: 25
Resta: 15
Multiplicación: 100
División: 4
El número mayor es: 20
```

Debes utilizar al menos un `IF / ELSE`.

Practica: variables de sustitución,operaciones matemáticas, `IF` y salida por consola.

Resultado

```plsql
DECLARE
    v_number_1 NUMBER(9) := &var_num_1;
    v_number_2 NUMBER(9) := &var_num_2;


BEGIN
    dbms_output.put_line('Número 1: ' ||v_number_1);
    dbms_output.put_line('Número 2: ' ||v_number_2);

    dbms_output.put_line('Suma: ' || (v_number_1 + v_number_2));
    dbms_output.put_line('Resta: ' || (v_number_1 - v_number_2));
    dbms_output.put_line('Multiplicación: ' || (v_number_1 * v_number_2));
    dbms_output.put_line('División: ' || (v_number_1 / v_number_2));
    IF v_number_1 > v_number_2 THEN
        dbms_output.put_line('El numero mayor es: ' ||v_number_1);
    ELSIF v_number_2 > v_number_1 THEN
        dbms_output.put_line('El numero mayor es: ' ||v_number_2);
    ELSE
        dbms_output.put_line('Ambos números son iguales');
    END IF;
END;
```

Ejercicio 3 — Sistema de evaluación

Crea un bloque anónimo que solicite una nota entre `1.0` y `7.0`.

El programa debe indicar:

```text
1.0 - 3.9 → Reprobado
4.0 - 4.9 → Aprobado
5.0 - 5.9 → Bueno
6.0 - 7.0 → Excelente
```

Además:

Si la nota está fuera del rango `1.0 - 7.0`, mostrar `"Nota inválida"`.
Utiliza `IF / ELSIF / ELSE`.
Luego realiza una segunda versión utilizando `CASE`.

Practica: entrada de datos,condiciones múltiples, `IF` y `CASE`.

Resultado

```plsql
DECLARE
    v_nota_1 NUMBER(9) := &var_nota_1;
    v_nota_2 NUMBER(9) := &var_nota_2;
    v_nota_promedio NUMBER(9, 2);
BEGIN
    v_nota_promedio := (v_nota_1 + v_nota_2)/2;
    dbms_output.put_line (v_nota_promedio);

    CASE
        WHEN v_nota_promedio < 4.0 THEN dbms_output.put_line ('Reprobado');
        WHEN v_nota_promedio < 5.0 THEN dbms_output.put_line ('Aprobado');
        WHEN v_nota_promedio < 6.0 THEN dbms_output.put_line ('Bueno');
        WHEN v_nota_promedio <= 7.0 THEN dbms_output.put_line ('Excelente' );
    END CASE;

END;
```

Clase 2 — RECORD

Ejercicio 4 — RECORD de estudiante

Crea un `RECORD` llamado `estudiante_record` que tenga:

```text
id
nombre
edad
nota
```

Crea una variable de ese tipo y asígnale valores.

Después muestra:

```text
ID: 1
Nombre: Carlos
Edad: 22
Nota: 6.4
```

Finalmente utiliza un `IF` para determinar si está aprobado.

Practica: `TYPE ... IS RECORD`, variables `RECORD`, atributos y acceso mediante `.`.

Resolución

```plsql
-- CONSULTA
DECLARE
    -- CREAMOS UN RECORD QUE ALMACENA LOS TIPOS DEFINIDOS
    TYPE estudiante_record IS RECORD(
        rut VARCHAR2(11),
        nombre VARCHAR2(30),
        edad NUMBER(2),
        nota NUMBER(2,1)
    );

    -- LOS TIPOS DE ESE RECORD SE LE ASIGNAN A LA VARIABLE, AHORA
    -- FUNCIONA COMO UN OBJETO EN OOP
    v_estudiante estudiante_record ;
BEGIN
    -- EN ESTE CASO,PARTICULAR,LE ASIGNAMOS LOS DATOS EN MEMORIA
    v_estudiante.rut := '18977594-6' ;
    v_estudiante.nombre := 'Jose' ;
    v_estudiante.edad := 31;
    v_estudiante.nota := 3.4 ;

    dbms_output.put_line ( 'RUT: ' ||v_estudiante.rut );
    dbms_output.put_line ( 'RUT: ' ||v_estudiante.nombre );
    dbms_output.put_line ( 'RUT: ' ||v_estudiante.edad );
    dbms_output.put_line ( 'RUT: ' ||v_estudiante.nota );

    IF v_estudiante.nota > 3.9 THEN
        dbms_output.put_line ('Aprobado');
    ELSE
        dbms_output.put_line ('Reprobado');
    END IF;
END;
```

Ejercicio 5 — RECORD + `%TYPE`

Supongamos que tienes una tabla:

```text
EMPLEADO
---------
ID_EMPLEADO
NOMBRE
SUELDO
```

Crea un `RECORD` cuyos atributos utilicen `%TYPE`:

```text
id → EMPLEADO.ID_EMPLEADO%TYPE
nombre → EMPLEADO.NOMBRE%TYPE
sueldo → EMPLEADO.SUELDO%TYPE
```

Luego asigna valores manualmente y muestra la información.

Después calcula un aumento del `10%` del sueldo y muestra:

```text
Empleado: Pedro
Sueldo actual: 600000
Sueldo con aumento: 660000
```

Objetivo adicional: comprender la diferencia entre declarar:

```PLSQL
NUMBER
```

y:

```PLSQL
EMPLEADO.SUELDO%TYPE
```

Practica: `RECORD`, `%TYPE`, acceso a atributos y operaciones.

```PLSQL
-- CREAMOS TABLA
CREATE TABLE EMPLEADO(
    ID_EMPLEADO VARCHAR2(10) PRIMARY KEY,
    nombre VARCHAR2(30) NOT NULL
);

-- ME FALTO UN DATO
ALTER TABLE EMPLEADO
ADD sueldo NUMBER(8) NOT NULL;

DECLARE
    -- DEFINIMOS UN RECORD CON TODOS LOS TIPOS DE LA TABLA CREADA
    TYPE empleado_record IS RECORD(
        emp_id EMPLEADO.ID_EMPLEADO%TYPE,
        emp_nombre EMPLEADO.nombre%TYPE,
        emp_sueldo EMPLEADO.sueldo%TYPE
);
    -- ALMACENAMOS EL RECORD EN UNA VARIABLE
    v_emp empleado_record;
BEGIN
    -- ASIGNAMOS VALORES
    v_emp.emp_id := '5' ;
    v_emp.emp_nombre := 'Juan' ;
    v_emp.emp_sueldo := 600000;

    dbms_output.put_line ( 'ID: ' ||v_emp.emp_id );
    dbms_output.put_line ( 'Nombre: '||v_emp.emp_nombre);
    dbms_output.put_line ( 'Sueldo: '||v_emp.emp_sueldo);
    dbms_output.put_line ( 'Sueldo ajustado: ' ||v_emp.emp_sueldo * 1.1);
END;
```

```HTML
Objetivo adicional: comprender la diferencia entre declarar:
NUMBER y EMPLEADO.SUELDO%TYPE

Respuesta: uno lo declaramos directamente, ya que la tabla no existe. Es una consulta independiente.
El otro,funciona en base a una tabla,y saca los tipos de datos de una fi la.
```

Ejercicio 6 — `%ROWTYPE` + `SELECT INTO`

Utiliza la tabla `EMPLEADO`.

Crea:

```PLSQL
v_empleado EMPLEADO%ROWTYPE;
```

Solicita mediante `&` un `ID_EMPLEADO`.

Luego realiza un `SELECT ... INTO ...` para obtener ese empleado completo.

Finalmente muestra sus datos.

Agrega una condición:

```text
Si sueldo >= 700000
    → "Empleado con sueldo alto"

Si no
    → "Empleado con sueldo estándar"
```

Practica: `SELECT INTO`, `%ROWTYPE`, acceso a columnas mediante `.`, `IF` y `DBMS_OUTPUT`.

RESPUESTA

```plsql
-- POBLAMOS DATOS EN TABLA EMPLEADO
INSERT INTO EMPLEADO(ID_EMPLEADO, NOMBRE, SUELDO)
VALUES ('3', 'JUAN', 600000);

INSERT INTO EMPLEADO(ID_EMPLEADO, NOMBRE, SUELDO)
VALUES ('5', 'PEDRITO', 7000000);

DECLARE
    v_empleado EMPLEADO%ROWTYPE;
    -- Crearemos una columna, que indica si el sueldo es alto o bajo
    -- Para eso, necesitamos declarar un nuevo atributo
    v_nivel_sueldo VARCHAR(30);
BEGIN
    -- LOS DATOS LOS INSERTAMOS EN NUESTRO RECORD
    SELECT ID_EMPLEADO, NOMBRE, SUELDO
    INTO v_empleado
    FROM EMPLEADO
    WHERE ID_EMPLEADO = &ID_EMPLEADO;

    IF v_empleado.sueldo >= 700000 THEN
        v_nivel_sueldo := 'ALTO';
    ELSE
        v_nivel_sueldo := 'BAJO';
    END IF;

    dbms_output.put_line('ID: '||v_empleado.id_empleado);
    dbms_output.put_line('Nombre: '||v_empleado.nombre);
    dbms_output.put_line('Sueldo: '||v_empleado.sueldo);
    dbms_output.put_line('Sueldo: '||v_nivel_sueldo);
END;
```

Clase 3 — Cursores

Ejercicio 7 — Cursor básico

Crea un cursor que consulte todos los empleados:

```plsql
SELECT *
FROM EMPLEADO
```

Utiliza:

```plsql
EMPLEADO%ROWTYPE
```

para almacenar cada fi la.

El cursor debe:

- Declararse.
- Abrirse.
- Realizar un `FETCH`.
- Comprobar `%NOTFOUND`.
- Mostrar el nombre del empleado.
- Repetirse hasta terminar los registros.
- Cerrarse.

La salida podría ser:

```text
Empleados:

Juan
Pedro
Ana
Carlos
Maria
```

No agregues todavía cálculos ni condiciones.

Objetivo: dominar la estructura básica de un cursor:

```text
DECLARE
    CURSOR
    RECORD

BEGIN
    OPEN
    LOOP
        FETCH
        EXIT WHEN
        ...
    END LOOP;
    CLOSE;
END;
```

Respuesta

```plsql
DECLARE
    CURSOR C_EMPLEADOS IS
        SELECT *
        FROM EMPLEADO
        ORDER BY  SUELDO DESC;

    v_empleado EMPLEADO%ROWTYPE;
BEGIN
    OPEN c_empleados;

    LOOP
        FETCH c_empleados INTO v_empleado;
        EXIT  WHEN c_empleados%NOTFOUND;
        dbms_output.put_line ( v_empleado.nombre );
    END LOOP;

    CLOSE c_empleados;
END;
```

Ejercicio 8 — Cursor + cálculo

Utiliza un cursor sobre `EMPLEADO`.

Por cada empleado debes mostrar:

```text
Empleado: Juan
Sueldo actual: 500000
Sueldo con aumento: 550000
```

El aumento será del `10%`.

Además, si el sueldo original es mayor o igual a `700000`, muestra:

```text
Categoría: Sueldo alto
```

En caso contrario:

```text
Categoría: Sueldo estándar
```

Debes combinar:

```text
CURSOR
%ROWTYPE
FETCH
%NOTFOUND
IF
operaciones matemáticas
DBMS_OUTPUT
```

Respuesta

```plsql
DECLARE
    CURSOR C_EMPLEADOS IS
        SELECT *
        FROM EMPLEADO
        ORDER BY  SUELDO DESC;

    v_empleado EMPLEADO%ROWTYPE;
BEGIN
    OPEN c_empleados;

    LOOP
        FETCH c_empleados INTO v_empleado;
        EXIT  WHEN c_empleados%NOTFOUND;
        dbms_output.put_line ( 'Empleado: '||v_empleado.nombre);
        dbms_output.put_line ( 'Sueldo actual: '||v_empleado.sueldo  );
        dbms_output.put_line ( 'Sueldo con aumento: '||v_empleado.sueldo   * 1.1 );

        IF  v_empleado.sueldo  >= 700000  THEN
            dbms_output.put_line ( 'Categoría: Sueldo alto');
        ELSE
            dbms_output.put_line ( 'Categoría: Sueldo estándar');
        END IF ;
    END LOOP;

    CLOSE c_empleados;
END;
```

Ejercicio 9 — Cursor + contador + acumulador

Utiliza un cursor para recorrer todos los empleados.

El programa debe calcular:

- Cantidad total de empleados.
- Suma de todos los sueldos.
- Sueldo promedio.
- Cantidad de empleados con sueldo mayor o igual a `700000`.
- Empleado con el sueldo más alto.

Al final debería mostrar algo parecido a:

```text
===== RESUMEN =====

Cantidad de empleados: 8
Suma de sueldos: 5.800.000
Sueldo promedio: 725.000
Empleados con sueldo alto: 4

Empleado con mayor sueldo:
Ana
Sueldo: 950.000
```

Para resolverlo necesitarás pensar en variables auxiliares como:

```text
contador
acumulador
promedio
mayor_sueldo
nombre_mayor_sueldo
contador_sueldo_alto
```

Estas variables deberán actualizarse mientras el cursor recorre cada fila.

Objetivo: comprender qué ocurre con cada registro durante la ejecución de un cursor y combinar cursores con lógica de programación.

Respuesta

```plsql
DECLARE
    CURSOR C_EMPLEADOS IS
        SELECT *
        FROM EMPLEADO
        ORDER BY  SUELDO DESC;

    v_empleado EMPLEADO%ROWTYPE;

    -- AUXILIARES
    contador  NUMBER(4)  := 0;
    acumulador NUMBER(10)  := 0;
    promedio NUMBER(10)  := 0;
    mayor_sueldo  NUMBER(10)  := 0;
    nombre_mayor_sueldo VARCHAR2(30);
    contador_sueldo_alto    NUMBER(10)  := 0;

BEGIN
    OPEN c_empleados;

    LOOP
        FETCH c_empleados INTO v_empleado;
        EXIT  WHEN c_empleados%NOTFOUND;

        -- Contar cantidad de empleados
        contador  := contador  + 1;

        -- Sumar total de sueldos
        acumulador := v_empleado.sueldo  + acumulador;

        -- PROMEDIO
        promedio := (acumulador /  contador);

        -- Empleados con sueldo alto
        IF  v_empleado.sueldo  >= 700000 THEN
            contador_sueldo_alto   := contador_sueldo_alto   + 1;
        END IF;

        -- EMPLEADO CON MAYOR SUELDO
        IF  v_empleado.sueldo  > mayor_sueldo THEN
            mayor_sueldo  := v_empleado.sueldo;
            nombre_mayor_sueldo := v_empleado.nombre;
        END IF;

    END LOOP;
        dbms_output.put_line ('===== RESUMEN =====');

        dbms_output.put_line ('Cantidad de empleados: '||contador );
        dbms_output.put_line ('Suma de sueldos: ' ||acumulador  );
        dbms_output.put_line ('Sueldo promedio: ' ||promedio  );
        dbms_output.put_line ('Empleados con sueldo alto: ' ||contador_sueldo_alto    );
        dbms_output.put_line ('Empleado con mayor sueldo: ' ||nombre_mayor_sueldo );
        dbms_output.put_line ('Sueldo: ' ||mayor_sueldo  );

    CLOSE c_empleados;
END;
```

Clase 3 — VARRAY

Ejercicio 10 — VARRAY básico

Crea un `VARRAY` con capacidad máxima para 5 números.

Inicialízalo con:

```text
10
20
30
40
50
```

Recorre el VARRAY utilizando un `FOR` y muestra:

```text
Elemento 1: 10
Elemento 2: 20
Elemento 3: 30
Elemento 4: 40
Elemento 5: 50
```

Practica: declaración de `VARRAY`,constructor,índices y `FOR`.

Respuesta

```plsql
DECLARE
    TYPE numeros IS VARRAY(5) OF NUMBER;

    v_numeros numeros;
BEGIN
    v_numeros := numeros(10, 20, 30, 40, 50);

    FOR i IN 1..v_numeros.COUNT LOOP
        dbms_output.put_line(v_numeros(i));

    END LOOP;
END;
```

Ejercicio 11 — VARRAY de nombres

Crea un `VARRAY` que pueda almacenar hasta 5 nombres.

Inicialízalo con:

```text
Juan
Pedro
Ana
Carlos
Maria
```

Recórrelo y muestra solamente los nombres que tengan más de 4 caracteres.

Resultado esperado:

```text
Pedro
Carlos
Maria
```

Debes combinar:

```text
VARRAY
FOR
IF
LENGTH()
```

Respuesta

```plsql
DECLARE
    TYPE nombres IS VARRAY(5) OF VARCHAR2(30);

    v_nombres nombres;
BEGIN
    v_nombres = nombres('Juan', 'Pedro', 'Ana', 'Carlos', 'María');

    FOR i IN 1..v_nombres.COUNT LOOP
        IF length(v_nombres(i)) > 4 THEN
            dbms_output.put_line(v_nombres(i));
        END IF;

    END LOOP;
END;
```

Ejercicio 12 — VARRAY + procesamiento

Crea un `VARRAY` de números con capacidad máxima para 10 elementos.

Comienza con:

```text
10
20
30
40
```

Luego utiliza `EXTEND` para agregar nuevos elementos y termina teniendo:

```text
10
20
30
40
50
60
```

Después recorre el VARRAY y calcula:

- Suma total.
- Promedio.
- Número mayor.
- Número menor.

Salida esperada:

```text
Suma: 210
Promedio: 35
Mayor: 60
Menor: 10
```

Objetivo: comprender la diferencia entre:

- Capacidad máxima del `VARRAY`.
- Cantidad actual de elementos.
- `EXTEND`.
- Acceso mediante índices.
- Recorrido de elementos.

Respuesta

```plsql
DECLARE
    TYPE numeros IS  VARRAY(10)  OF NUMBER;

    v_num numeros;
    suma_total  number := 0;
    contador  NUMBER := 0;
    promedio NUMBER := 0;
    numero_mayor NUMBER := 0;
    numero_menor NUMBER := 100;
BEGIN
    v_num = numeros(10,  20,  30,  40);
    v_num.EXTEND(2);
    v_num(5)  := 50;
    v_num(6)  := 60;


    FOR i  IN  1..v_num.COUNT LOOP
        -- CONTADOR
        contador  := contador  + 1;

        -- SUMA TOTAL
        suma_total  := v_num(i)  + suma_total;

        -- PROMEDIO
        promedio  := (suma_total/contador);

        -- NUMERO MAYOR
        IF  v_num(i)  > numero_mayor THEN
            numero_mayor := v_num(i);
        END IF;

        -- NUMERO MENOR
        IF  v_num(i)  < numero_menor THEN
            numero_menor := v_num(i);
        END IF;
    END LOOP;

    dbms_output.put_line ('Suma: ' ||suma_total  );
    dbms_output.put_line ('Promedio: ' ||promedio);
    dbms_output.put_line ('Mayor: ' ||numero_mayor);
    dbms_output.put_line ('Menor: ' ||numero_menor);

END;
```

Orden recomendado

Se recomienda resolver los ejercicios en este orden:

```text
CLASE 1
│
├── Ejercicio 1 → Fácil
├── Ejercicio 2 → Medio
└── Ejercicio 3 → Medio+
        │
        ▼
CLASE 2 — RECORD
│
├── Ejercicio 4 → Fácil
├── Ejercicio 5 → Medio
└── Ejercicio 6 → Medio+
        │
        ▼
CLASE 3 — CURSORES
│
├── Ejercicio 7 → Fácil
├── Ejercicio 8 → Medio
└── Ejercicio 9 → Difícil
        │
        ▼
CLASE 3 — VARRAY
│
├── Ejercicio 10 → Fácil
├── Ejercicio 11 → Medio
└── Ejercicio 12 → Difícil
```

Forma de trabajo

La idea es resolver cada ejercicio sin mirar una solución completa.

Cuando haya dudas:

1. Intentar plantear primero la estructura.
2. Escribir el código.
3. Ejecutarlo en Oracle.
4. Revisar el error si aparece.
5. Compartir el código para revisarlo.
6. Corregirlo entendiendo por qué falló, en lugar de reemplazarlo directamente por una solución.

El objetivo no es solamente conseguir que el código funcione,sino poder explicar qué hace cada parte del bloque PL/SQL.

Conceptos que se repasan

Clase	Conceptos
Clase 1	Bloques anónimos,variables, `DBMS_OUTPUT`,variables de sustitución,operaciones, `IF`, `CASE`, ciclos
Clase 2	`RECORD`, `%TYPE`, `%ROWTYPE`,atributos, `SELECT INTO`
Clase 3	Cursores explícitos, `OPEN`, `FETCH`, `%NOTFOUND`, `CLOSE`, `%ROWTYPE`
Clase 3	`VARRAY`,constructor,índices, `FOR`, `EXTEND`, procesamiento

Progresión general

```text
Variables
    ↓
Condiciones
    ↓
RECORD
    ↓
%TYPE
    ↓
%ROWTYPE
    ↓
SELECT INTO
    ↓
Cursores
    ↓
Recorrer múltiples filas
    ↓
Contadores y acumuladores
    ↓
VARRAY
    ↓
Procesamiento de estructuras
```